Unit IV – Linked List

Prof. Pranjal Pandit

Unit IV – Linked List

1. Static vs dynamic memory allocation;
2. Introduction to linked lists,
3. Realization using dynamic memory management,
4. Linked list as ADT.
5. Types: singly linked (linear and circular),
6. Doubly linked,
7. Doubly circular linked lists.
8. Primitive operations: create, traverse, search, insert, delete, sort, concatenate.
9. Polynomial manipulation using linked lists;
10. Generalized Linked List (GLL) concept and polynomial representation using GLL.

Static vs dynamic memory allocation

Memory allocation means **reserving memory space in RAM for storing data during program execution**.

1. Static Memory Allocation: The memory size is decided **before or at compile time** and generally cannot be changed during program execution.

int marks[5];

2. Dynamic Memory Allocation

In **dynamic memory allocation**, memory is allocated **during program execution (runtime)**.

The required amount of memory can be decided based on the user's input or program requirements.

In C++, dynamic memory is commonly allocated using:

new

and released using:

delete

```
#include <iostream>
using namespace std;
int main()
{
int n;
cout << "Enter number of students: ";
cin >> n;
                                        int *marks = new int[n];
for(int i = 0; i < n; i++)
{
cout << "Enter marks: ";
cin >> marks[i];
}
cout << "\nMarks are:\n";
for(int i = 0; i < n; i++)
 cout << marks[i] << " ";
                                                delete[] marks;
return 0;
}
```

Dynamic single variable

int *p = new int;

Store a value: *p = 50;

Release memory: delete p;

Dynamic array

int *arr = new int[n];

Release dynamic array: delete[] arr;

Remember the difference:

```
delete p;       // single variable

delete[] arr;   // array
```

Static vs Dynamic Memory — Simple Comparison

Feature	**Static Memory Allocation**	**Dynamic Memory Allocation**
Allocation time	Compile time / fixed-size declaration	Runtime
Memory size	Fixed	Can be decided at runtime
Flexibility	Low	High
Memory location	Usually stack/static storage depending on declaration	Heap
Keyword	Normal declaration	`new`
Deallocation	Usually automatic	Programmer-controlled with `delete`
Example	`int a[10];`	`int *a = new int[n];`
Size change	Not possible for the existing array	Can allocate a new block of different size
Efficiency	Generally faster/simple	Slight overhead for allocation
Suitable when	Size is known beforehand	Size is unknown or variable

Linked Lists — Introduction

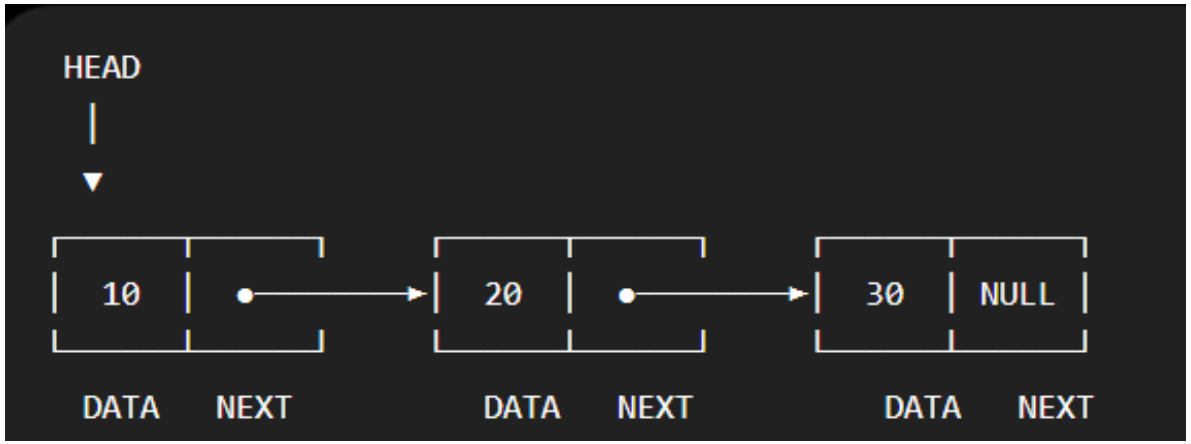
- Linked lists are one of the most important **dynamic data structures**.
- They are especially **useful when the number of elements is not known** in advance and frequent insertion/deletion operations are required.
- A linked list is **a linear data structure** in which elements are stored in separate memory locations called **nodes**.
- Unlike an array, the nodes of a linked list **do not have to be stored in consecutive memory locations**.

Each node generally contains two parts:

- **Data** — stores the actual value.
- **Link/Pointer** — stores the address of the next node.

Basic structure of a node

For example, if we want to store: 10, 20, 30



The **HEAD** pointer stores the address of the first node.

The last node contains:

NEXT = NULL

which indicates the end of the linked list.

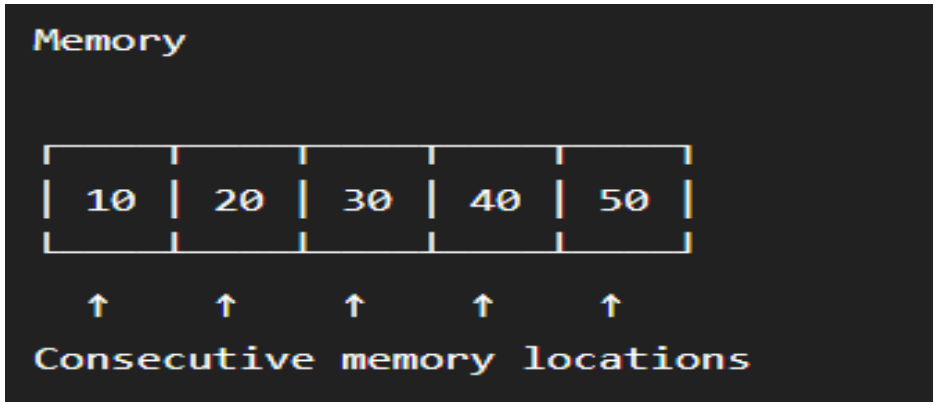
Why is it called a "Linked" List?

The nodes are connected using pointers.

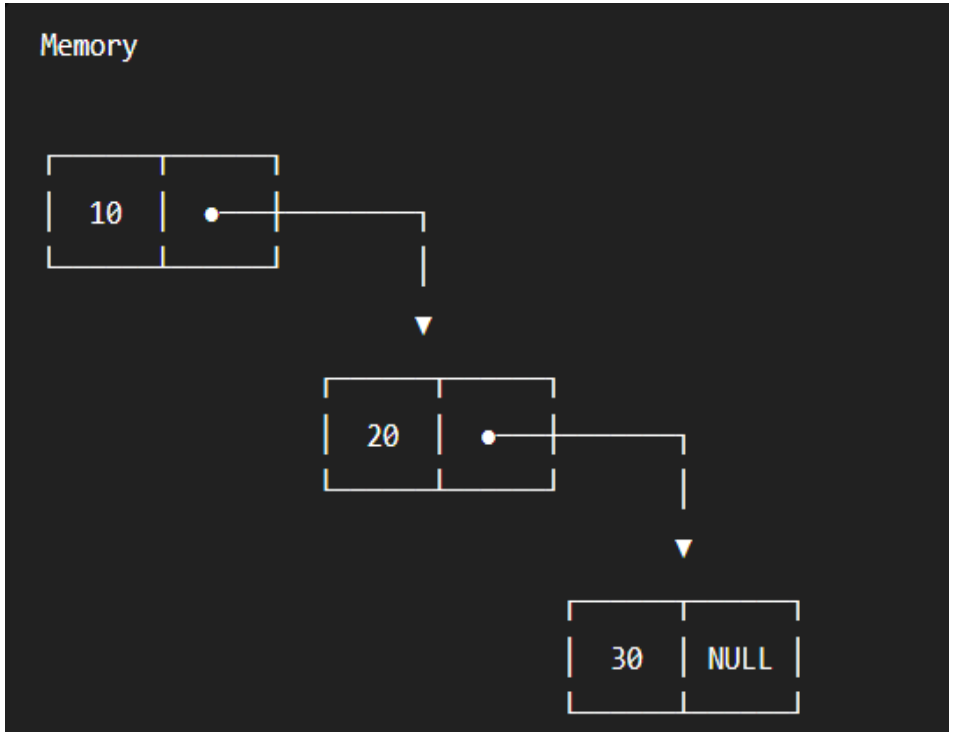


The pointer in each node establishes the connection to the next node.

Array vs Linked List



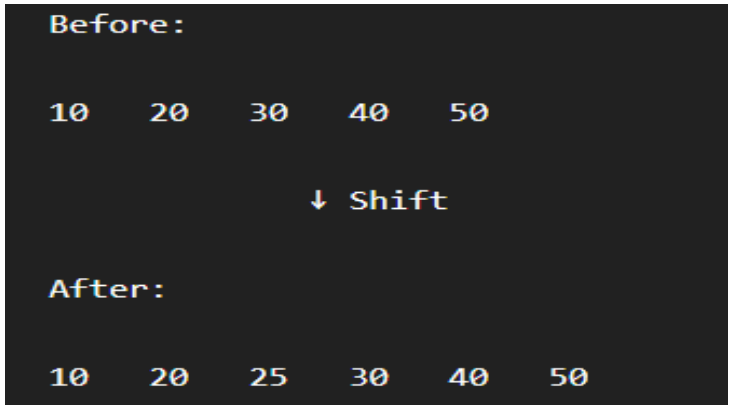
An array normally requires a contiguous block of memory.



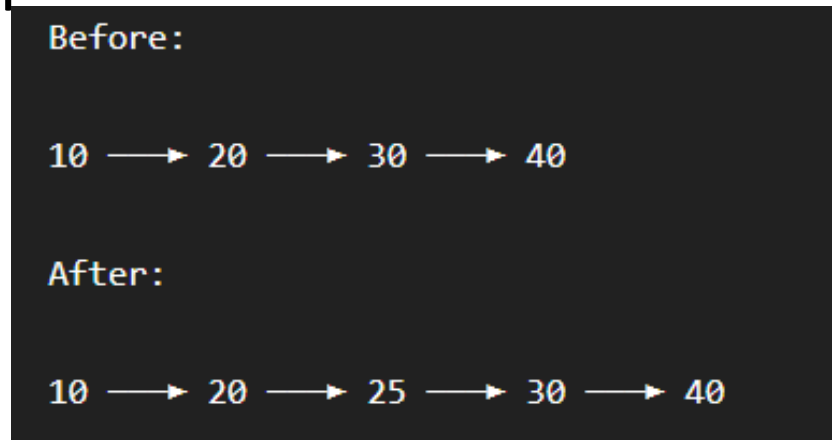
The nodes can be located at different memory addresses.

Why Do We Need Linked Lists?

Extremely useful when there are frequent insertion and deletion. In array we have to shift elements in every insertion and deletion.



In linked Lists we just need to change the pointer locations. The nodes themselves do not need to be shifted.



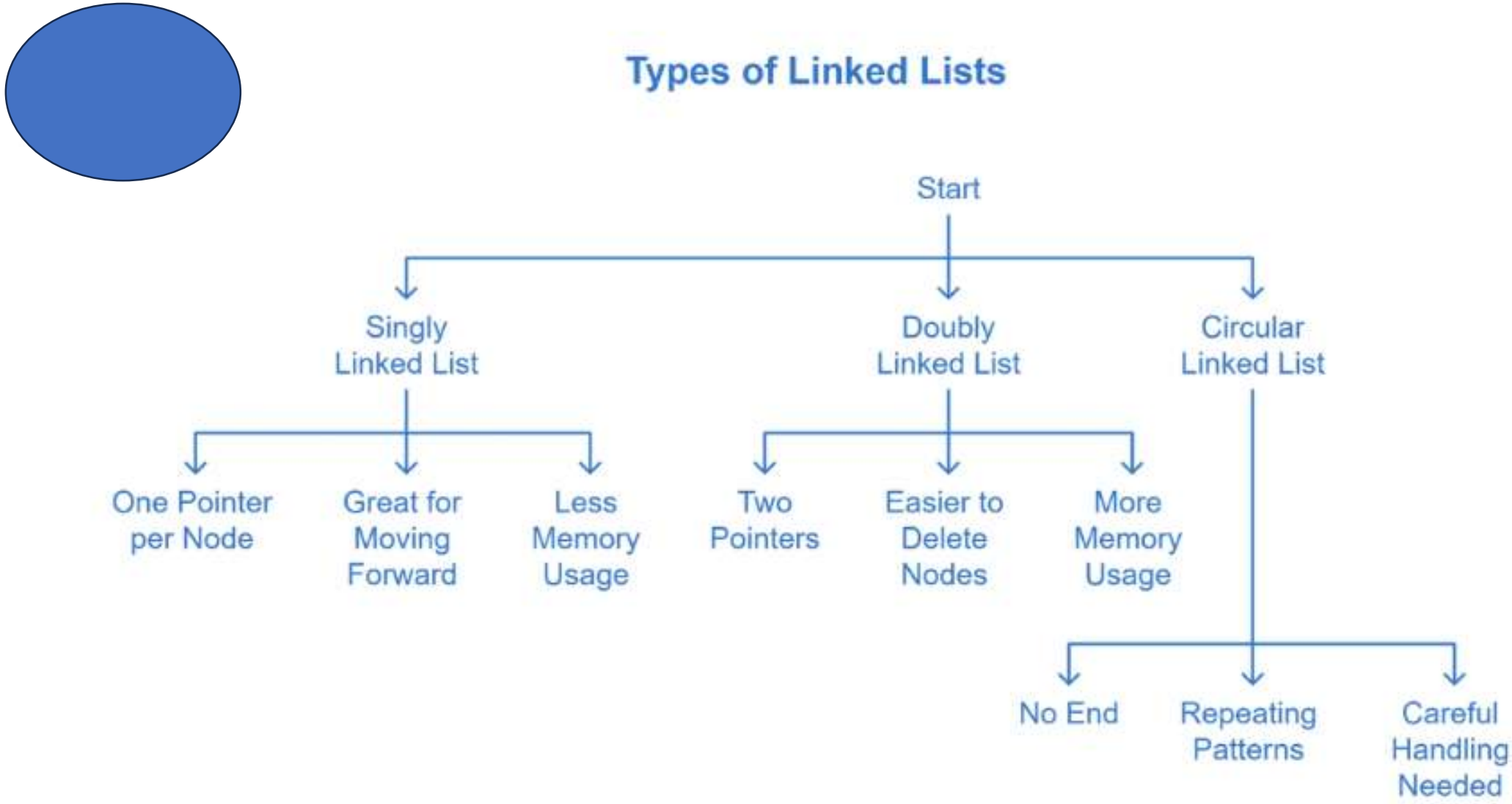
Why linked list data structure needed?

- **Dynamic Data structure:** The size of memory can be allocated or de-allocated at run time based on the operation insertion or deletion.

- **Ease of Insertion/Deletion:** The insertion and deletion of elements are **simpler than arrays since no elements need to be shifted after insertion and deletion**, Just the address needed to be updated.

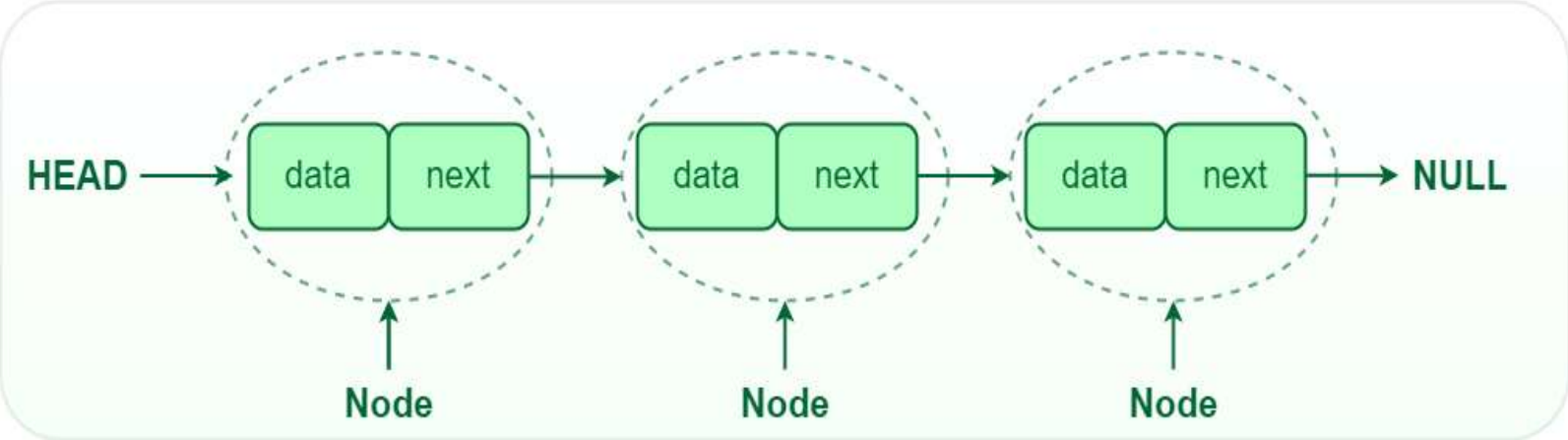
- **Efficient Memory Utilization:** As we know Linked List is a dynamic data structure the size increases or decreases as per the requirement so this avoids the wastage of memory.

- **Implementation:** Various advanced data structures can be implemented using a linked list like a stack, queue, graph, hash maps, etc.


Types of Linked Lists
Start
Singly Linked List
Doubly Linked List
Circular Linked List
One Pointer per Node
Great for Moving Forward
Less Memory Usage
Two Pointers
Easier to Delete Nodes
More Memory Usage
No End
Repeating Patterns
Careful Handling Needed

Singly Linked List

A singly linked list consists of nodes where every node points to the **next node**.



Each node can be represented as:

```
struct Node
{
        int data;
        Node *next;
};
```

```cpp
// Definition of a Node in a singly linked list
struct Node {

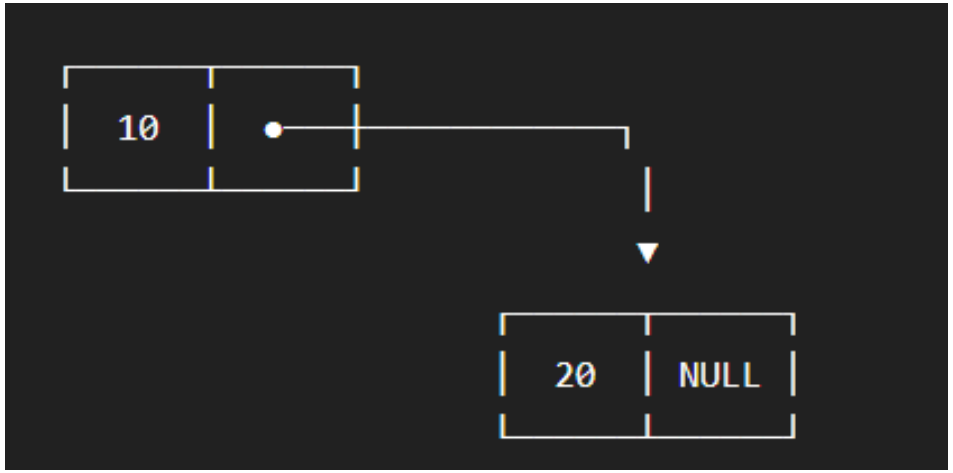
    // Data part of the node
    int data;

    // Pointer to the next node in the list
    Node* next;

    // Constructor to initialize the node with data
    Node(int data) {
        this->data = data;
        this->next = nullptr;
    }
};
```

Why Does a Linked List Use Dynamic Memory?

- Initially: HEAD = NULL
- When we need a node: Node *newNode = new Node;
- When another element is required, another node can be created.



- Traversal of items can be done in the **forward direction only** due to the linking of every node to its next node.

Creating a Node Dynamically

```
struct Node
{
   int data;
   Node *next;
};
```

Now create a node: Node *newNode = new Node;

Then assign data: newNode->data = 10;
And initialize the link: newNode->next = NULL;

Complete code:

```
Node *newNode = new Node;

newNode->data = 10;
newNode->next = NULL;
```

```
#include <iostream>
using namespace std;

// Define a node
struct Node
{
    int data;
    Node* next;
};

int main()
{
    // Initially, linked list is empty
    Node* head = NULL;

    // Create first node
    Node* newNode = new Node;
    newNode->data = 10;
    newNode->next = NULL;
```

```
 // Make first node the head
    head = newNode;

    // Create second node
    newNode = new Node;
    newNode->data = 20;
    newNode->next = NULL;

    // Connect first node to second node
    head->next = newNode;

    // Display the linked list
    Node* temp = head;

    while (temp != NULL)
    {
        cout << temp->data << " ";
        temp = temp->next;
    }

    return 0;
}
```

```
struct Node {
    int data;
    Node* next;
};

int main() {

    Node* head = new Node();

    head->data = 10;
    head->next = new Node();

    head->next->data = 20;
    head->next->next = new Node();

    head->next->next->data = 30;
    head->next->next->next = nullptr;

    // Display
    Node* temp = head;

    while (temp != nullptr) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL";

    return 0;
}
```

```
struct Node
{
    int data;
    Node* next;
};
int main()
{
    Node* head = nullptr;
    Node* temp = nullptr;
    // CREATE 10 NODES
    for (int i = 1; i <= 10; i++)
    {
        int value;
        cout << "Enter value " << i << ": ";
        cin >> value;
        // Create new node
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = nullptr;
```

```
// First node
if (head == nullptr)
{
    head = newNode;
    temp = newNode;
}
else
{
    temp->next = newNode;
    temp = newNode;
```

```
// DISPLAY LINKED LIST
cout << "\nLinked List: ";
temp = head;
while (temp != nullptr)
{
    cout << temp->data << " -> ";
    temp = temp->next;
}
cout << "NULL\n";
// INSERT A NODE AT END
int value;
cout << "\nEnter value to insert at the end: ";
cin >> value;
Node* newNode = new Node;
newNode->data = value;
newNode->next = nullptr;
```

```
// Find last node
temp = head;
while (temp->next != nullptr)
{
    temp = temp->next;
}
temp->next = newNode;
```

```
// DISPLAY AFTER INSERTION
    cout << "\nLinked List after insertion: ";
    temp = head;
    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL\n";
    return 0;
}
```

Commonly used operations on Singly Linked List:

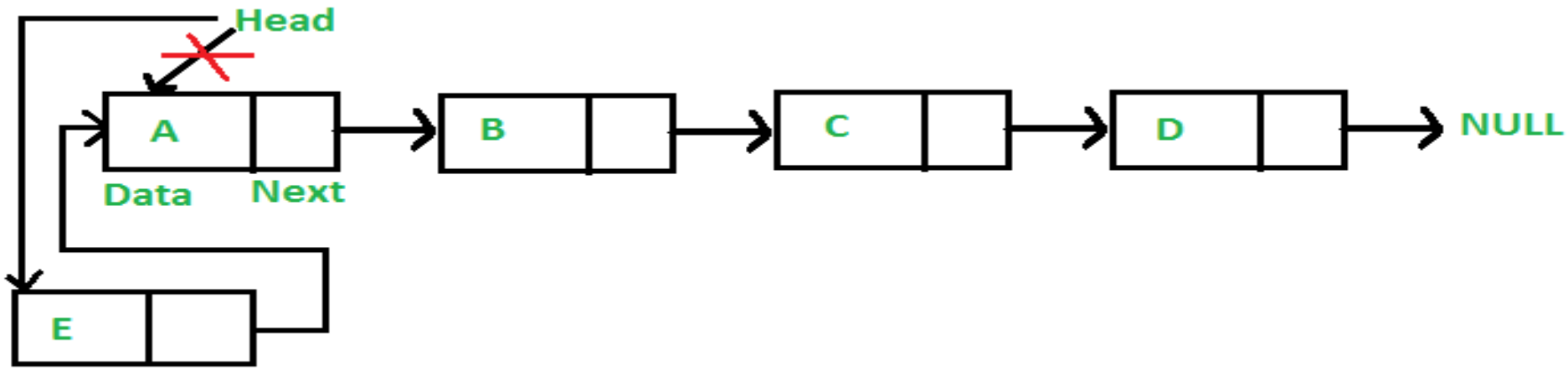
The following operations are performed on a Single Linked List

- **Insertion**
- **Deletion**
- **Searching**
- **Updation**
- **Traversal**
- **Sorting**
- **Concatenation**

Insertion

1. Insert at the front
2. Insert in between
3. Insert at the end

To insert a node at the start/beginning/front of a Linked List, we need to:

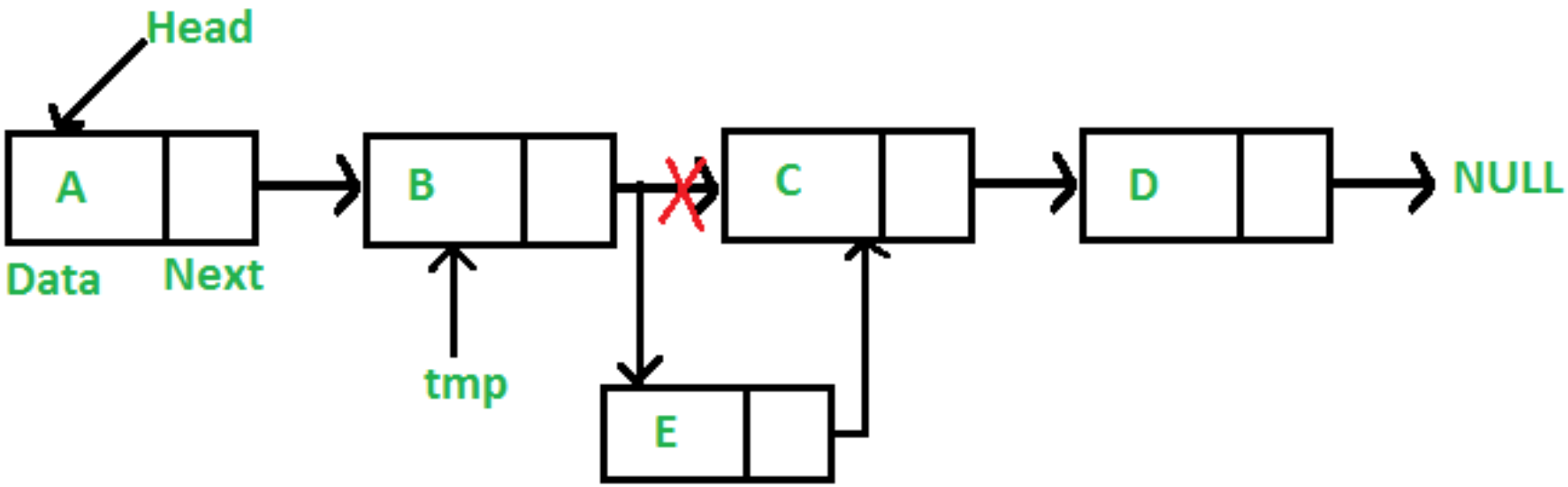


Implementation:

Given a reference (pointer to pointer) to the head of a list and an int, inserts a new node on the front of the list.

```
void push(Node** head_ref, int new_data)
{
        Node* new_node = new Node();
        new_node->data = new_data;
        new_node->next = (*head_ref);
        (*head_ref) = new_node;
}
```

To insert a node after a given node in a Linked List, we need to:

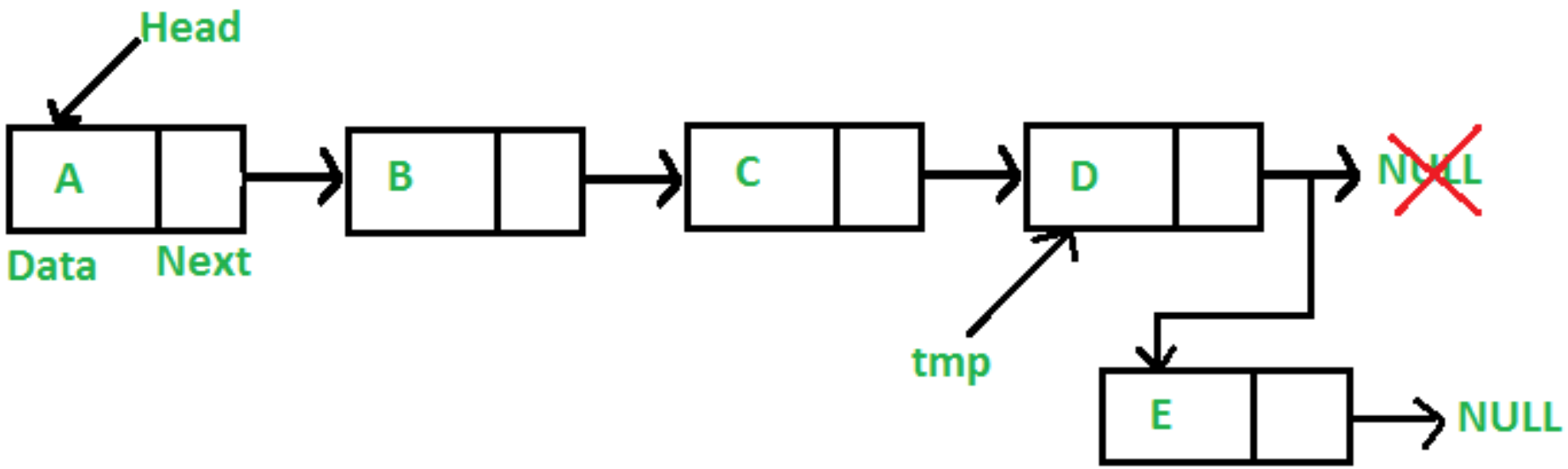


Implementation:

Given a node prev_node, insert a new node after the given prev_node

```
void insertAfter(Node* prev_node, int new_data)
{
    if (prev_node == NULL) {
            cout << "The given previous node cannot be NULL";
            return;
    }
    Node* new_node = new Node();
    new_node->data = new_data;
    new_node->next = prev_node->next;
    prev_node->next = new_node;
}
```

To insert a node at the end of a Linked List, we need to:



Implementation:

Given a reference (pointer to pointer) to the head of a list and an int, appends a new node at the end

```
void append(Node** head_ref, int new_data)
{
        Node* new_node = new Node();
        Node* last = *head_ref;
        new_node->data = new_data;

        new_node->next = NULL;
        if (*head_ref == NULL) {
                *head_ref = new_node;
                return;
        }
        while (last->next != NULL) {
                last = last->next;
        }

        last->next = new_node;
        return;
}
```